Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Distributed Systems and Applications

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

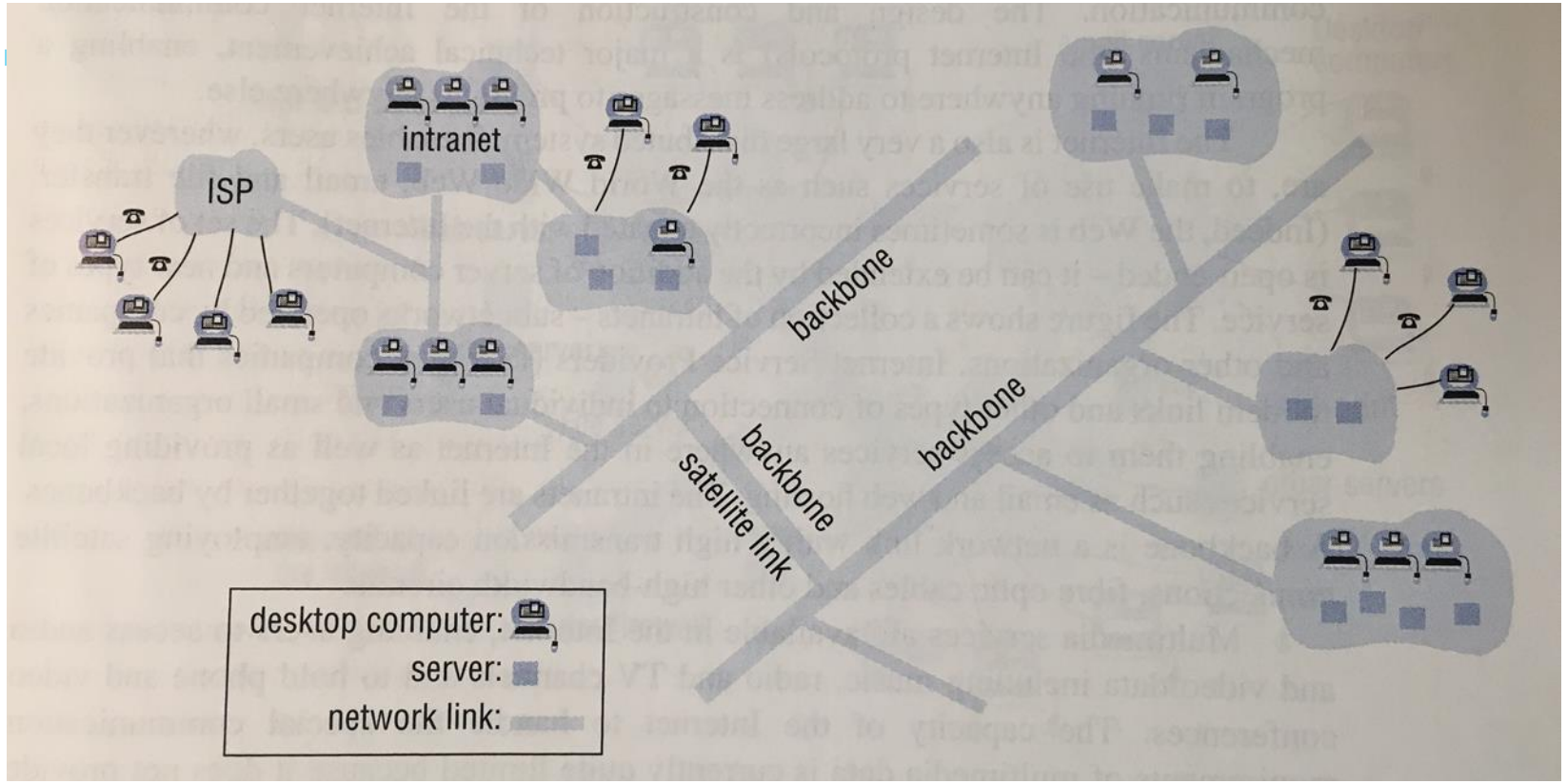
- Types of Distributed System (e.g. Internet/intranets)
- Ubiquitous computing
- Key concepts and characteristics of Distributed Computing
  - These are concepts that we will return to in detail later on in the lecture series
- A primer on architectures for the next session

Distributed Systems

- Any network-connected group of machines can be considered a distributed system.
- Typically, distributed systems exploit the use of their interconnectedness to share data and resources.
- Not all machines need to be the same in a distributed system, so a single very powerful server can share its resources and data with lots of low-power devices.
- Some machines may be specialist (e.g. printer/router/email server) and others generic (e.g. PC, smart phone)

UNIVERSITY of HULL

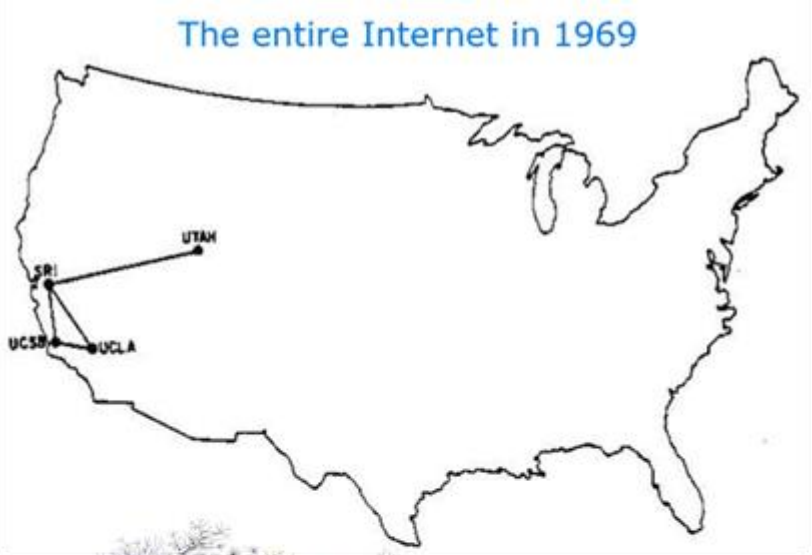
Distributed Systems

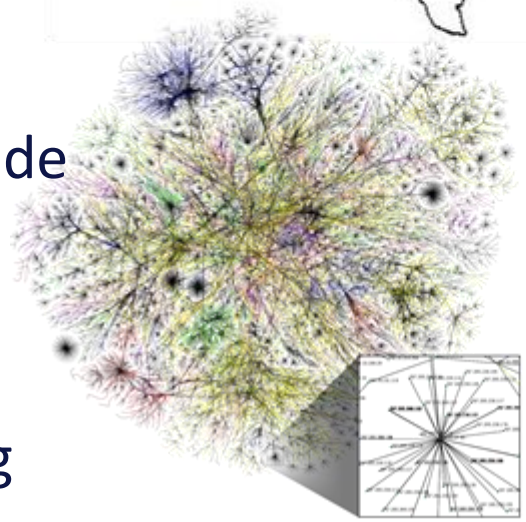


UNIVERSITY of HULL

The Internet

- The Internet is (mostly) comprised of a number of Internet Service Providers (ISPs) who provide connections to users.
- These ISPs are linked by a backbone. The backbone is a network of high-capacity transmission methods. In most cases, the backbone is not a single link but a web to provide redundancy.
- Various content is available on the Internet and there are a huge number of specifications for sending and receiving this content.

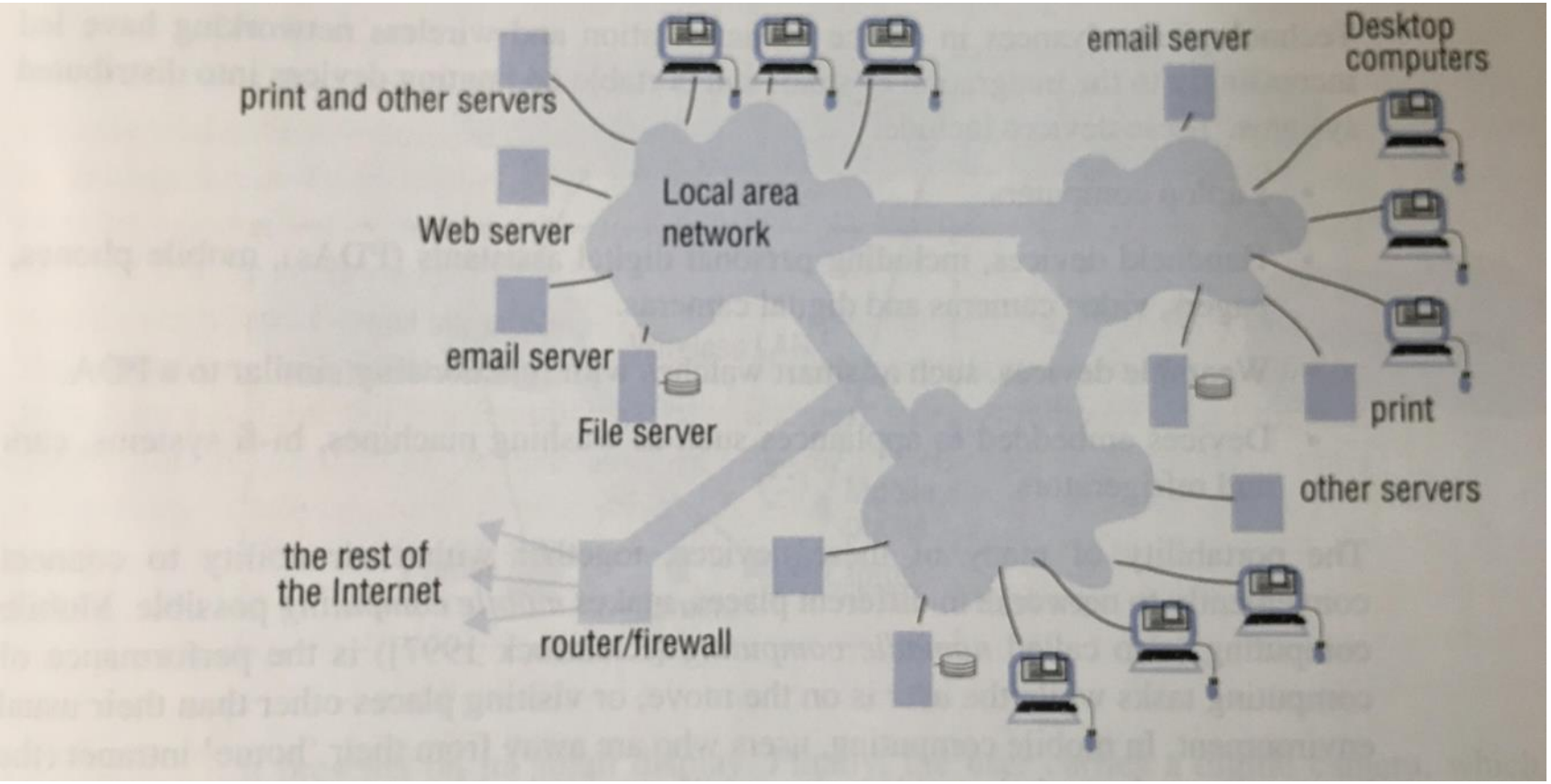




Modern visualisation

UNIVERSITY of HULL

Distributed Systems



UNIVERSITY of HULL

Intranets

- Intranets are portions of the internet that are separately administered - often to enforce local security policies
- Intranets consist of one or more local area networks (LANs) linked by backbone connections. They can exist in a single building or over a much larger area
- Intranet is connected to internet via router; is protected by a firewall
- Traffic may be managed very differently if it has come from within the intranet vs. from the Internet
- Some intranets are not connected to the internet at all

UNIVERSITY of HULL

Mobile and Ubiquitous Computing

- The goal for Ubiquitous Computing is to fit computers into all aspects of our everyday life so seamlessly that we may forget that we are interacting with computers at all.
- A key concept in ubiquitous computing is that these devices are connected together, allowing smart decisions to be made through a more pervasive ambient intelligence.
- This means that services and devices are able to be designed which we interact with, without having to rely on a conventional user interface.
- Examples include:
  - A connected fridge and cupboard that are able to determine what food provisions you have at home, partner this with a recipe and automatically order all of the ingredients you need in the quantities you need them.

- We are already seeing a fast trend towards mobile devices and wearables as well as the Internet of Things (IoT)
- There is a huge amount of research into this: big data, connected devices, information processing, security…

UNIVERSITY of HULL

Distributed Applications

- Distributed applications make use of distributed systems to spread execution over multiple machines.

- These applications may:
  - simply use similar resources on lots of machines to make tasks faster
  - use the specific features or resources of a dedicated machine to accomplish tasks that would otherwise be difficult or costly for a generic machine
  - make use of distributed systems to request and share data across multiple machines

Distributed Applications

- Banking, shopping and payment systems
  - Internet and mobile banking
  - On-line shops
- Multiplayer online games
- Real-time process control
  - Aircraft control systems
  - Industrial control systems
- Social media
- Distributed databases
- Scientific computing
- Distributed software development support
  - Distributed version control systems
  - Distributed continuous integration systems

UNIVERSITY of HULL

Why Use a Distributed Application?

- Efficiency
  - Resource sharing
  - Reliability
  - Fault tolerance
- Integration
  - Integrating legacy systems
  - Different interfaces and data models
- Communication and synchronisation
  - Synchronisation (functionality, data)
- Geographical and technical constraints
  - Different locations
  - Different clients (e.g. for web browsers, mobile devices)

Example Distributed Application

- Consider a simple game app on your phone
- How does the app store and load high scores?

- Perhaps locally…
- Increasingly it’s done on both the device AND the web
- This means you can compare with friends/people you meet online/everyone

- The app looks and feels like a single application but really there is a service running on a server somewhere **maintaining** and **distributing** scores.

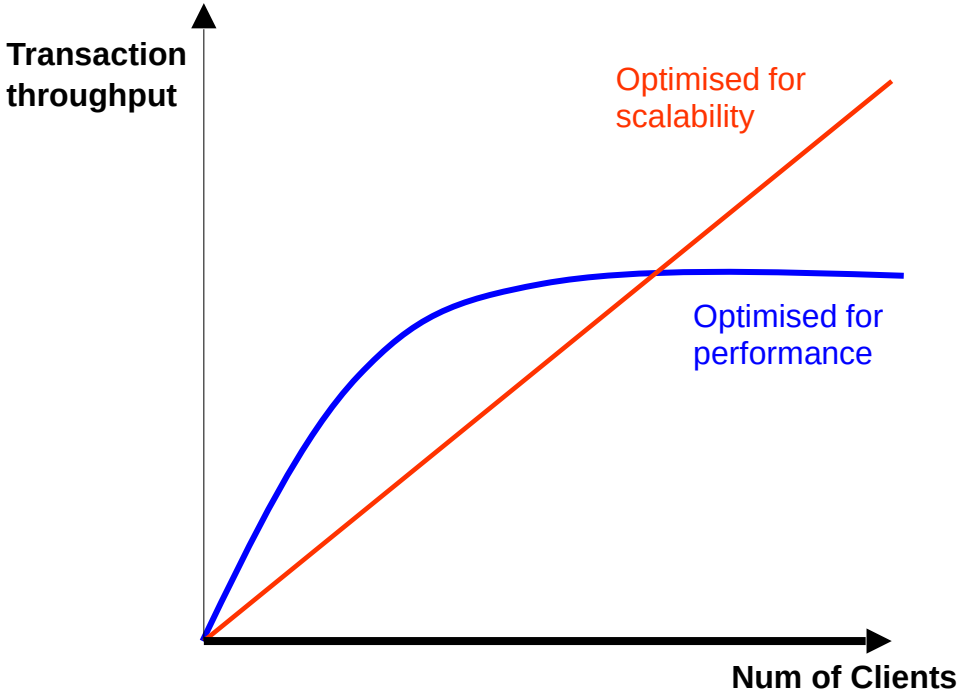
UNIVERSITY of HULL

Characteristics of Distributed Applications

- Execution spread over *more than one computer*
- Components are *networked*
- Different *failure modes*
  - failures are often *partial* – i.e. only one of the required components may fail whilst others continue operating. This is different to non-distributed applications
  - failure may not be due to failure of components but *failure of the network connection*
- Application appears as single *coherent whole*
  - user is not aware of the various networked machines (*transparency*)
- *Concurrency* is the norm in networked systems
  - servers interact with multiple clients
  - multiple clients may access a resource simultaneously
  - clients use multi-threading to maintain responsiveness

Performance and Scalability

- Performance
  - Measure of application speed
- Scalability
  - How speed varies as client load increases or server hardware is upgraded
- Performance and scalability are related; we can optimise for one or the other



UNIVERSITY of HULL

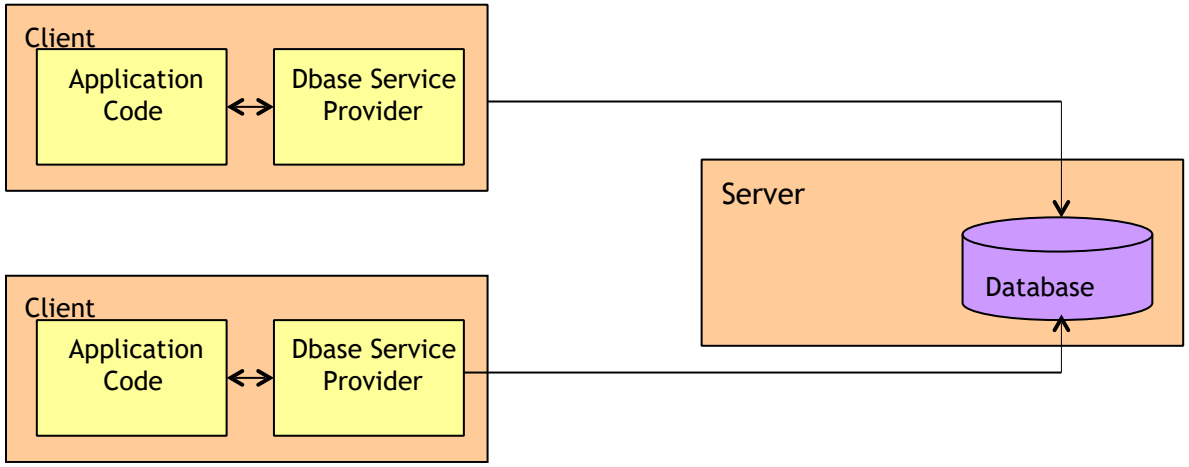
Performance and Scalability

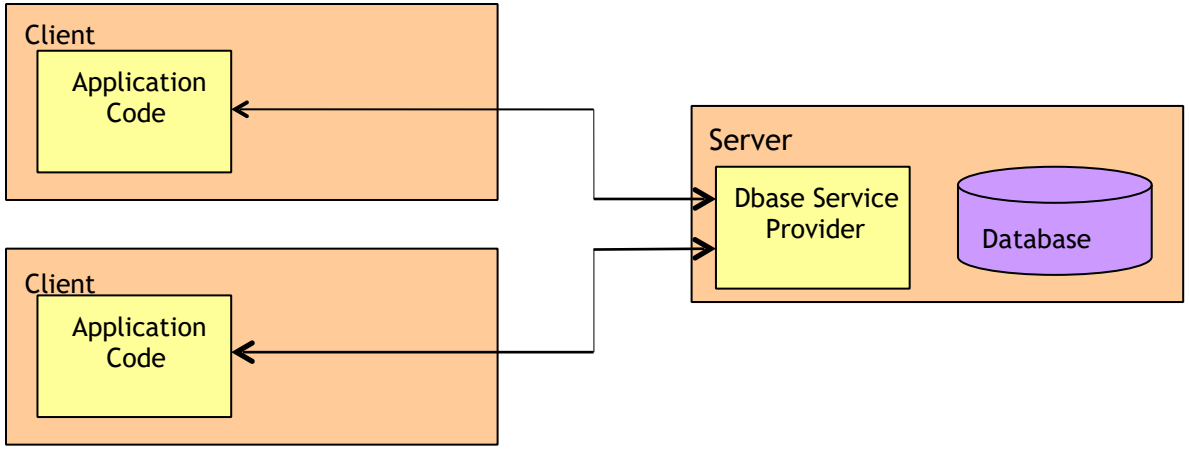
Which one is better for a large number of users?

Which one costs less to run?

Which one is more scalable?

Which is more maintainable?





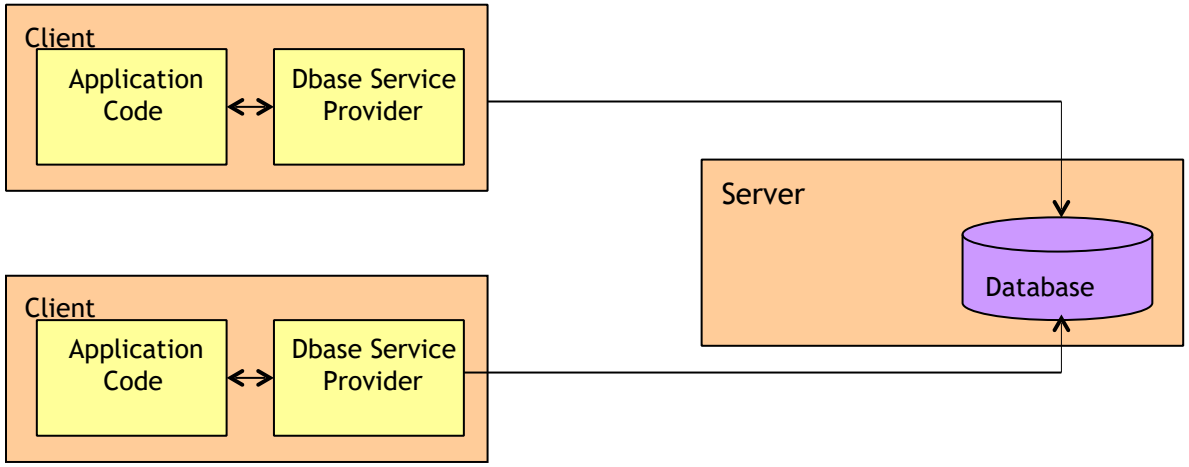
UNIVERSITY of HULL

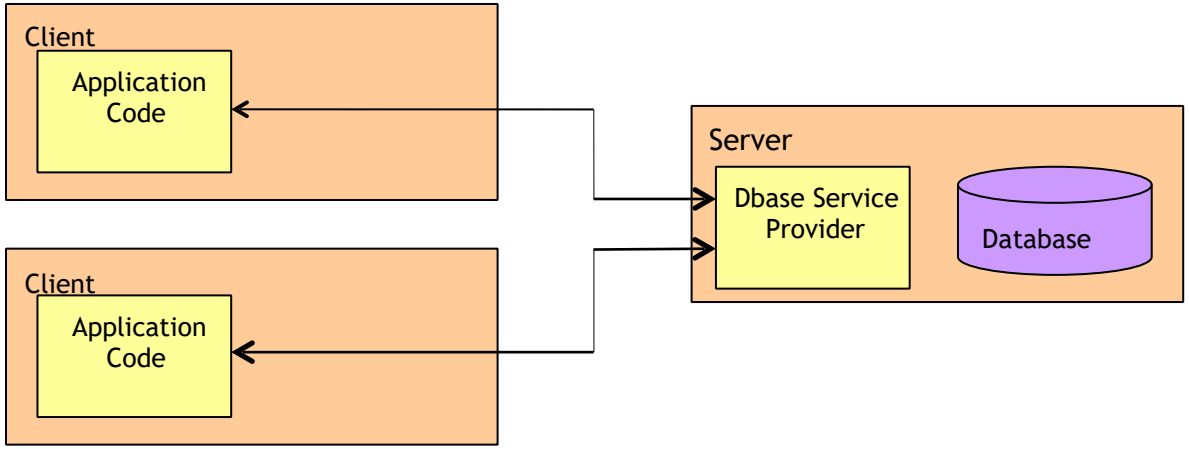
Performance and Scalability

One key thing to recognise is the challenge of **sync** between clients.

Database access can be very difficult to sync so the safest option is usually to manage it on the server.

But, by moving calculation to the server we also increase the server process load.

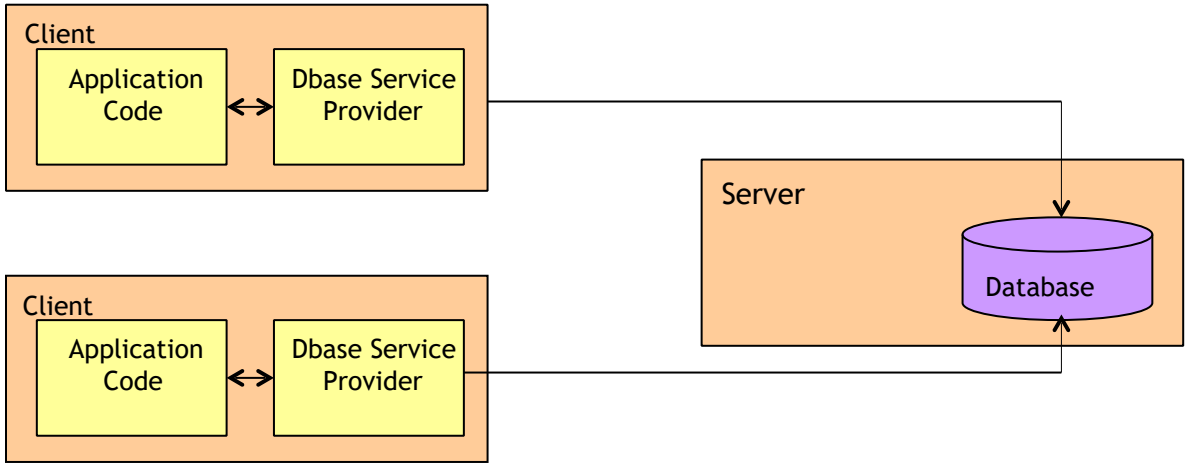




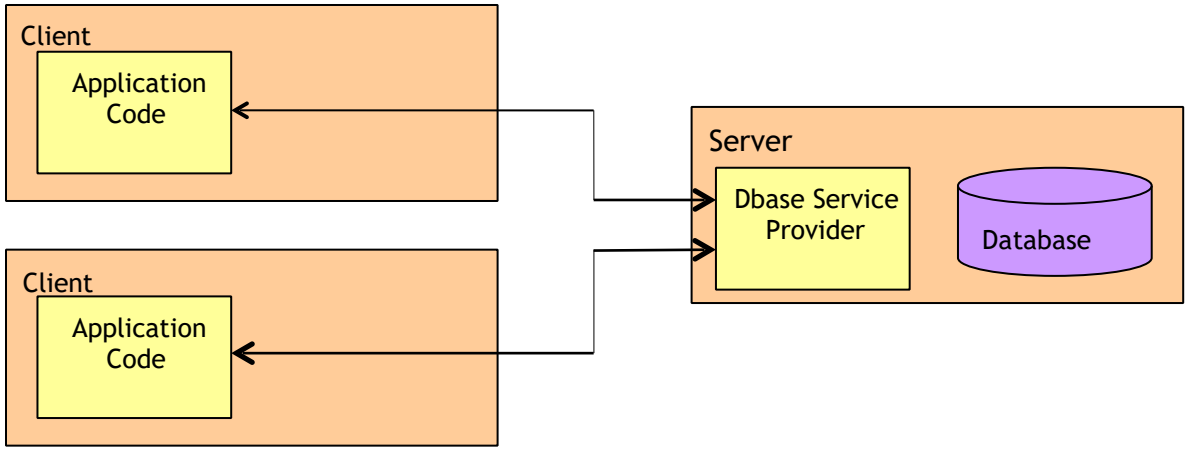
UNIVERSITY of HULL

Performance and Scalability

Best for performance with small number of clients. Minimal server-side slowdown.



Best for scalability; large number of clients but server does more (needs to be more powerful) and therefore costs more



UNIVERSITY of HULL

Balancing Scalability and Performance

- Typically, distributed systems require scalability and performance in different measures
- A system is *scalable* if it remains effective when number of resources/users increases significantly
- Scalability is a key criteria of many distributed applications
- Scalable system design requires:
  - Controlling cost of physical resources
    - possible to extend system at reasonable cost
  - Controlling performance loss
    - max. performance loss should be less than O(*log n*)
    - hierarchical structures better than linear structures
  - Preventing resource exhaustion
  - Avoiding performance bottlenecks
    - decentralization, replication and caching

UNIVERSITY of HULL

Security

- User privacy and security of data and resources is of paramount importance
- Key aspects
  - Confidentiality
  - Integrity
  - Availability
  - Authentication / Authorization
  - Nonrepudiation / Auditing
- Security is particularly important for mobile devices, which are easily lost or stolen
  - A secured corporate network can easily be compromised by the loss or theft of a single mobile device
- The network is fundamentally insecure

Concurrency

- Concurrency = doing several things at once

- Concurrency is the norm in networked systems
  - servers interact with multiple clients
  - multiple clients may access a resource simultaneously
  - clients may access multiple resources simultaneously
- Concurrency needs to be controlled to ensure
  - resources are shared correctly (and fairly?)
  - all clients have a *consistent* view of the state of system objects and resources
  - desired throughput and performance are achieved
- Concurrency control mechanisms and *transactions* can help
  - monitors, semaphores, locks, mutex's, etc

How can we achieve concurrency?

Multithreading – putting separate work onto separate cores OR switching threads on a single core to create simultaneous-like behaviours
Distribution / Scale Out – moving work onto another physical machine and to have multiple machines doing work at the same time

UNIVERSITY of HULL

Distributed Architectures

The principal distributed architectures are:

- **Client/Server**
  - Most common distributed architecture Servers provide services; Clients make requests of servers to use services. Server may act as a hub.
- **Peer-to-Peer**
  - All components are treated similarly; no automatic distinction between peers (e.g. no designated client or server) although peers might agree on (often temporary) relationships, roles and synchronisation.
- **Multi-tier (e.g. microservice)**
  - Generally a combination of client/server and peer-to-peer where a single access point or controller (e.g. a server) distributes tasks to multiple subsequent peers or servers, which may then distribute tasks further where required.
- **Mobile code**
  - Distributed components not run remotely but downloaded and used on demand

Questions

- What are the differences between distributed applications and client-server applications?
- Does making an application distributed automatically make it scalable?
- Is a peer-peer architecture limited to two devices?

UNIVERSITY of HULL

Questions

- What are the differences between distributed applications and client-server applications?

Distributed applications are any application which uses more than one concurrent piece of software to perform tasks. Often this execution is spread across remote machines.

A client-server model can be thought of as one form of distributed application, but in many cases most of the application exists and executes on *either* the client *or* the server (normally the client does most of the work).

Questions

- Does making an application distributed automatically make it scalable?

Definitely not. Distributed applications normally have to overcome a number of physical (hardware) and digital (software/code) limitations before any level of scalability is afforded.

It is completely possible to write a distributed application that acts as if it were a single-threaded program running on a single computer – in this case it is not scalable at all.

Questions

- Is a peer-peer architecture limited to two devices?

Perhaps this is a matter of semantics. A peer-to-peer architecture at it's most basic is two connected devices. Typically these devices share a more balanced responsibility of the overall system.

However, a larger architecture (for example a ring) is simply comprised of a number of peer-to-peer connections. Similarly, a huge number of devices with multiple peer-to-peer connections could create a mesh.

We'll look at these architectures later in the course.

Reflection

We've looked at:

- Types of Distributed System (e.g. Internet/intranets)
- Ubiquitous computing
- Key concepts and characteristics of Distributed Computing
- A primer on architectures for the next session

In the next session we will continue to examine distributed architectures and look, in detail, at the Client-Server model

UNIVERSITY of HULL